



Technical Report

Smart IoT Cane for Obstacle Detection and Emergency Monitoring

Prototyping in the Internet of Things

SoSe 2026

Ammar Shahzad – N°1808919 and Diogo Falcão - N°1926316

Index

1. Introduction	3
1.1 Motivation and Project objective	3
1.2 Overview of the proposed solution	3
2. Concept and Use Cases	4
2.1 Project Idea.....	4
2.2 Use Cases	4
3. System Architecture.....	4
4. Hardware Design	5
4.1 Components.....	5
4.2 Sensor Placement	6
4.3 Electronics.....	7
4.4 Breadboard prototype.....	8
4.5 Wiring & Crimping	9
4.6 Pin assignments.....	9
4.7 Materials and Fabrication.....	10
5. Software Implementation.....	11
5.1 ESP32 Firmware	12
5.2 Fall Detection Algorithm.....	13
5.3 MQTT Communication.....	14
5.4 Mosquitto Configuration:	14
5.5 Web Application	14
6. Development Process	16
7. Challenges and Limitations	17
8. Future Improvements.....	18
9. Conclusion	18
Bibliography.....	19

1. Introduction

1.1 Motivation and Project objective

In the context Prototyping Internet of Things, we aimed to develop a device that is both useful and relevant for everyday use. We got to the conclusion that we wanted to do something related to helping people. After brainstorming, we chose to develop a solution that could meaningfully support people in their daily routines and not help them rarely. Our initial concept was a smart cane for individuals with visual impairments. But during the ideation phase we realized that our target audience could be much bigger, and therefore, we could help a lot more people. So, we adapt the idea of the cane for a wider range of users: seniors, individuals with neurological conditions, people recovering from trauma or surgery, and more. This smart cane idea was also envisioned to be something cheap, easy to manufacture and suitable for mass production.

The main objective of our project is to develop a prototype of a smart cane that helps users move around more safely and comfortably by integrating sensors and microcontrollers. This objective is in line with the *World Health Organization (2024)*, which recognizes assistive technology as an essential factor in enabling people who are different abled to live more independently and participate fully in society.

1.2 Overview of the proposed solution

The proposed smart cane consists of multiple sensors enclosed in a custom 3D printed cane, connected to a microcontroller. The device communicates with a message broker, which relays real time data to a web application. This application allows authorized individuals such as the user, a caretaker, or family member, to monitor relevant information about the cane's usage and the user's condition.

2. Concept and Use Cases

2.1 Project Idea

Through the available materials in FabLab, sensors which could be used in this project were shortlisted. These sensors needed to detect nearby objects, detect slippery or/and wet floors, detect possible falls while using the cane, online dashboard with the possibility of notifying a caregiver remotely.

2.2 Use Cases

The Smart Cane is designed to assist users during everyday mobility while also improving their safety. Some common use cases include walking indoors like detecting walls, furniture, doors, or other obstacles and warning the user through different buzzer patterns. Other scenario would be walking outdoors in the assist of the user by detecting obstacles such as poles, bins, or other objects in the walking path. Use cases also include detecting areas with increased moisture, such as puddles or other slippery surfaces (e.g., waste), allowing the user to avoid them. One import use case would be the detection of fall in emergencies. If the cane remains stationary for an extended period, the system sends an SOS alert to the monitoring dashboard, allowing a caregiver to respond quickly.

3. System Architecture

Figure 1 presents the overall architecture of the Smart Cane system. The ESP8266 microcontroller collects data from the connected sensors and connects to a Wi-Fi network. It publishes sensor readings and SOS alerts to an Eclipse Mosquitto MQTT broker running inside a Docker container using the MQTT protocol. The frontend, developed with Next.js, React, and TypeScript, subscribes to these MQTT topics through WebSockets to receive data in real time. The processed information is then displayed on a web dashboard, allowing a caregiver to monitor the user's status and receive emergency alerts.

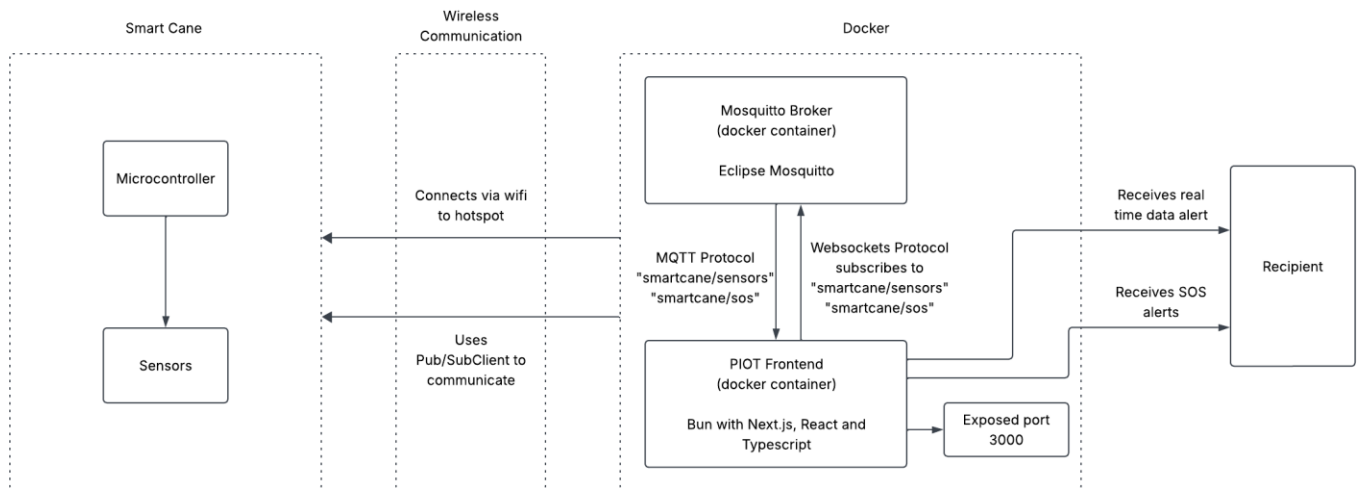


Image 1 – System Architecture

4. Hardware Design

4.1 Components

Component	Purpose
ESP32	Main controller
KY-032 IR sensors (×3)	Obstacle detection
KY-015 humidity sensor	Wet floor detection
KY-017 tilt switch	Fall detection
KY-006 passive buzzer	Audio feedback

Table 1 – Components and purpose

In total, there are six sensors used, as it is possible to see in table one. Next there are the sensors descriptions and their functionality in our prototype:

- IR sensor receiver and transmitter (KY-032) – these sensors were intended to detect nearby obstacles and warn the user. Our initial idea was to use three IR receivers (KY-022) together with three IR transmitters (KY-005), positioning one pair to monitor each direction of movement: front, left, and right. However, we

later replaced them with three KY-032 obstacle avoidance sensors because they were more suited for this task.

- Passive buzzer module (KY-006) – this sensor would play a sound if the IR sensors detected some obstacle in the way.
- Temperature and humidity sensor (KY-015) – This sensor measures temperature and humidity. In theory, we planned to use it to detect nearby wet surfaces by identifying localized changes in humidity that could indicate the presence of moisture.
- Tilt switch (KY-017) – This sensor is a vibration sensor, so, this means that is made for only detecting if there is some movement in a certain axis. If the sensor is placed horizontally, it can only tell if there is motion mostly in that direction. This was the most closely related sensor we could use to tell if there was a fall while the user was using the cane. Later we will address how we overcome this shortcoming.

Even though we already incorporated six sensors into the system, we initially considered adding vibration motors, LEDs, and buttons to further enhance the user experience (vibration feedback, LED to draw attention of people nearby and one button to turn on and off the cane and an SOS button). However, due to space, hardware, and implementation constraints, we had to prioritize the most essential components and were unable to include these additional features. The Z5903XIII vibration motors required separate power source to work which would have made our prototype complex and heavy. Nevertheless, we believe the final prototype provides a solid and well-balanced solution that successfully meets the project's main objectives.

4.2 Sensor Placement

Regarding the sensor placement, the humidity sensor is positioned at the bottom of the cane, as this location is closest to the ground and better suited to detecting higher humidity levels that may indicate wet surfaces. Directly above are the three IR obstacle detection sensors, oriented to monitor the front, left, and right directions relative to the user's movement. We decided not to include a fourth rear-facing sensor because it could continuously detect the user holding the cane rather than actual nearby obstacles. The ESP

microcontroller is mounted in the middle section of the cane. At the top of the cane are the buzzer, and the tilt switch, mounted horizontally to detect changes in the cane's orientation.

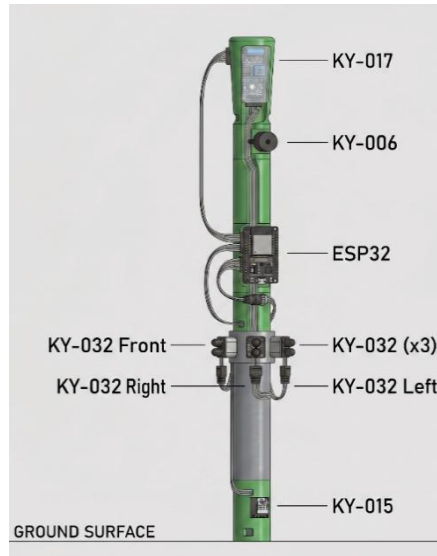


Image 2 – Cane Sensors

4.3 Electronics

The electronics of the smart cane are built around an ESP32 microcontroller, chosen primarily for its ease of availability and suitability for integrating multiple sensors. The ESP32 serves as the central processing unit, reads input from sensors and controls their outputs. There is a common ground and power line that goes down the hollow cane, this is done to minimize design and hide loose wires. But because of time constraints, the bottom part has loose hanging wires that are bound with tapes. All sensors share a common ground and power line and all are parallel. Together, this hardware configuration enables the cane to monitor the user's immediate environment and convey critical information through simple, intuitive signals.

From a hardware integration perspective, the electronics are assembled using jumper wires and crimped connections, which allows rapid prototyping and ease of change if anything goes south. At this prototype stage, the cane is powered directly from a laptop, which simplifies development and testing but does not yet include dedicated power management features such as batteries.

Similarly, no additional electrical protection components have been incorporated, as the focus is on validating the concept and basic functionality before optimizing robustness, safety, and energy efficiency in future iterations.

4.4 Breadboard prototype

Before integrating the electronics into the smart cane, a complete breadboard prototype was assembled using the ESP32 and the same set of KY sensors. This setup was used to test sensor functionality, validate the wiring, and debug the code before jumping onto 3d printing. During this phase, no significant wiring issues or functional problems were encountered, other than occasional reverse connections. As a result, no major changes to the electronic design were necessary, and the successful breadboard tests provided confidence that the system would perform reliably once integrated into the 3D printed cane.

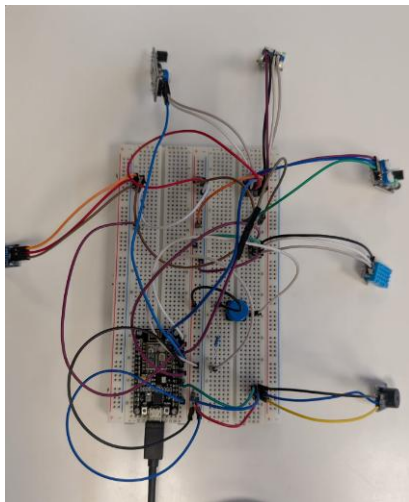


Image 4 – Sensors and breadboard



Image 5 – Process of crimping

4.5 Wiring & Crimping

The wiring and crimping of this prototype were implemented using 26 AWG wires combined with male and female crimp terminals along with Dupont connectors. All input connections between the ESP32 and the sensors were made via jumper wires, while ground and power connections were made with the help of common ground and power running down the hollow cane. The connections for ground and power were crimped to save space. Nearby sensors ground and power pins were crimped together to form a common pin: the three IR sensors and the humidity sensor pins were combined, and likewise the buzzer and tilt sensor share common supply lines. To keep the assembly compact, all wires were routed through the hollow shaft of the cane and fixed with tape at the end. Ground wires were color coded green, live (power) wires brown, and signal inputs blue to simplify assembly and debugging. The crimps were made manually, and due to inexperience around 25% of them had to be redone, with one problematic connection ultimately being reinforced with tape. Although this approach introduced some loose wiring at the end, it was chosen over soldering because of flexibility required during prototyping.

4.6 Pin assignments

The prototype uses six ESP8266 GPIO pins: D1 for the tilt switch, D2 for the front IR sensor, D3 and D4 for the left and right IR sensors, D5 for the humidity sensor, and D7 for the buzzer.

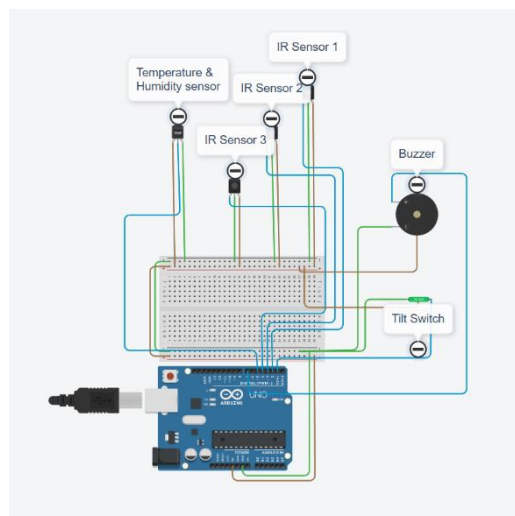


Image 6 – Circuit Diagram

4.7 Materials and Fabrication

PETG was chosen over PLA for the smart cane's mounts and enclosure due to its mechanical properties. The *young's modulus* of PLA is higher than PETG which makes it stiffer and brittle, that makes it less suitable for our project. PETG also has higher temperature resistance, preventing warping or softening when used outdoors under direct sunlight. Besides this, PETG provides better long term durability against mechanical strain and a much higher impact resistance, guaranteeing the enclosure absorbs shocks and bumps without cracking or shattering like brittle PLA. Even though PETG provided worse surface finish, it was good enough for our use case. The fabrication of all 3D printed components, including both the failed prototype parts and the final cane, required approximately half a spool of PETG filament.

4.8 CAD Design

The smart cane was designed in Fusion 360 with focus on rapid prototyping. The CAD model consists of three main sections, with the middle section designed to be printed multiple times to increase the overall length of the cane. The CAD model was specifically designed for Fused Deposition Modeling (FDM) 3D printing and was printed in FAB LAB. This model features a hollow interior that allows cables to be routed between the electronic components. The IR sensors and humidity sensor are integrated into the bottom section to detect obstacles and wet surfaces, while the buzzer and tilt switch are housed in the middle section. At this prototype stage, the microcontroller is externally attached to the cane using adhesive tape, prioritizing functionality over aesthetics, and ease of maintenance. The design process was iterative and required revisions to achieve correct dimensions and reliable threaded connections between the printed parts. The overall geometry was inspired by existing cane designs but changes were necessary to fit our use case. CAD design can be seen in figure.

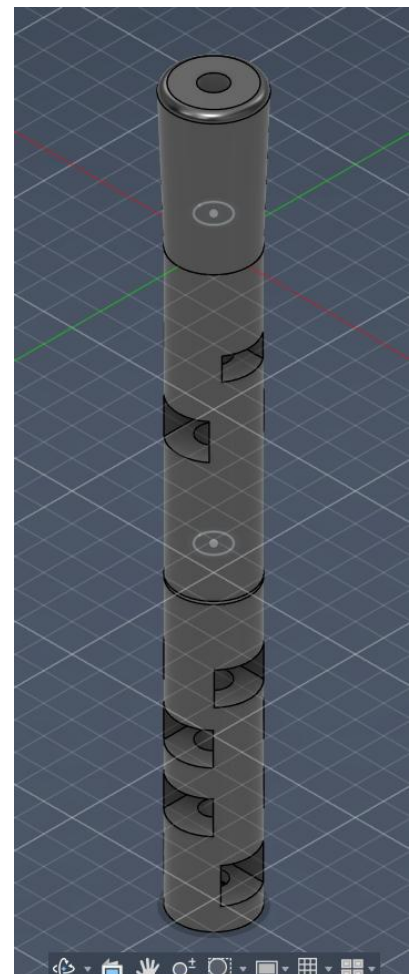


Image 7 – Cane CAD

4.9 Slicer

The G&M code file was prepared using PrusaSlicer and printed on a Prusa Core One printer with a 0.4 mm nozzle. PETG filament was selected due to its good mechanical strength as discussed above. The default print profile with a 0.20 mm speed setting was used, combined with a 15% infill density to achieve fast printing time. A brim was added to improve bed adhesion and reduce the risk of warping during printing. No support structures were required due to the orientation and geometry of the designed parts. The complete cane assembly, consisting of three parts, was printed simultaneously with a total printing time of approximately two hours. During the printing process, we encountered several issues, including unsuccessful threaded connections, printer calibration problems, and design dimension errors. These challenges required adjustments to both the CAD model and printing setup to improve the accuracy and functionality of the final prototype.

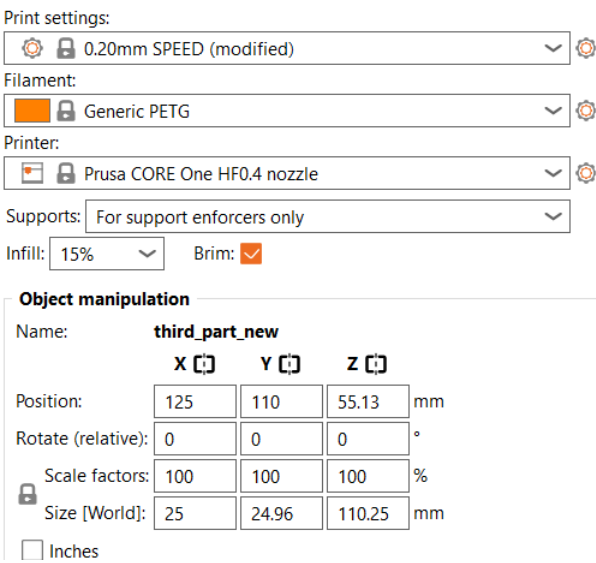


Image 8 – PrusaSlicer CAD settings

5. Software Implementation

The software architecture is divided into the microcontroller firmware, the message broker, and the web-based dashboard. The system utilizes the Arduino IDE for hardware programming and Next.js with Bun for the frontend.

5.1 ESP32 Firmware

The firmware was developed in C and C++. It uses the standard library WiFi for ESP32 and for network connectivity, PubSubClient for MQTT communication, and the DHT library for reading the DHT11 humidity and temperature sensor. The main loop performs the following tasks:

- **Network Management:** Continuously checks and reconnects to the Wi-Fi and MQTT broker if the connection drops.
- **Sensor Polling:** Reads digital inputs (IR sensors, tilt switch) and analog data (DHT11) every one second.
- **Data Publishing:** Packages the sensor states into a JSON payload and publishes it to the MQTT broker.
- **Obstacle Warning:** Triggers the buzzer based on proximity feedback.

To help the cane user understand where an obstacle is located, the system uses different beep patterns and frequencies:

Sensor	Feedback
Front IR (D2)	1 beep (500 Hz)
Right IR (D3)	2 beeps (800 Hz)
Left IR (D4)	3 beeps (1000 Hz)

Table 2 – Buzzer sensor feedback

```

if (ir1 == LOW) {
  beepCount = 1; frequency = 500;
} else if (ir2 == LOW) {
  beepCount = 2; frequency = 800;
} else if (ir3 == LOW) {
  beepCount = 3; frequency = 1000;
}

if (beepCount > 0) {
  for (int i = 0; i < beepCount; i++) {
    tone(Buzzer_Pin, frequency, buzzerDuration);
    delay(buzzerDuration + 100);
  }
}

```

Image 9 – Buzzer feedback logic

5.2 Fall Detection Algorithm

The fall detection logic is designed to trigger an emergency response if the cane detects a horizontal orientation without subsequent recovery. This is the pseudocode for better understanding:

```

IF TiltSwitch == HIGH (Cane is horizontal)
  Start 30-second Timer

  WHILE Timer < 30 seconds
    IF TiltSwitch == LOW (Cane is upright again)
      Cancel Alarm
      EXIT

  // If timer expires and cane is still
  horizontal
  Publish SOS Event to MQTT
  Trigger Emergency UI State

```

Image 10 – Fall Detection Pseudocode

Note: During development, the SOS trigger was simulated every 2 minutes via a timer to test the frontend response reliably. Relying solely on a basic digital tilt switch means the

system cannot differentiate between a cane that was accidentally dropped and a user who has actually fallen. Additionally, resting the cane flat on a table could trigger false positives.

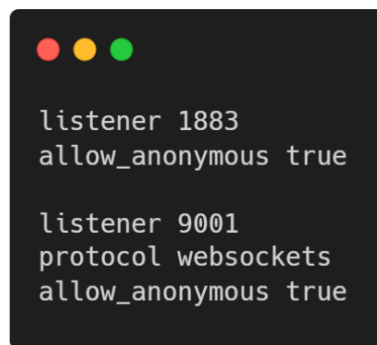
5.3 MQTT Communication

Message routing is handled by an Eclipse Mosquitto broker containerized via Docker. The broker is configured to accept standard MQTT traffic from the hardware and WebSocket connections from the frontend.

Topics Used:

- smartcane/sensors: Receives the combined JSON payload (temperature, humidity, tilt, IR states) every second.
- smartcane/sos: Receives emergency events when a fall is detected.

5.4 Mosquitto Configuration:



```
listener 1883
allow_anonymous true

listener 9001
protocol websockets
allow_anonymous true
```

Image 11 – MQTT Mosquitto configuration

5.5 Web Application

The monitoring dashboard is a React application built with Next.js, TypeScript, and Tailwind CSS. It runs in a Docker container powered by the Bun runtime. The frontend uses the mqtt library to subscribe to smartcane/sensors over WebSockets (port 9001) and updates digital and analog UI cards. It also subscribes to smartcane/sos. When

this is triggered, the UI instantly switches to an emergency state which has a acknowledgment button to clear the alarm and a direct "Call 911" action button. We also display a real-time console of incoming JSON payloads. Down below, it is possible to see the frontend:

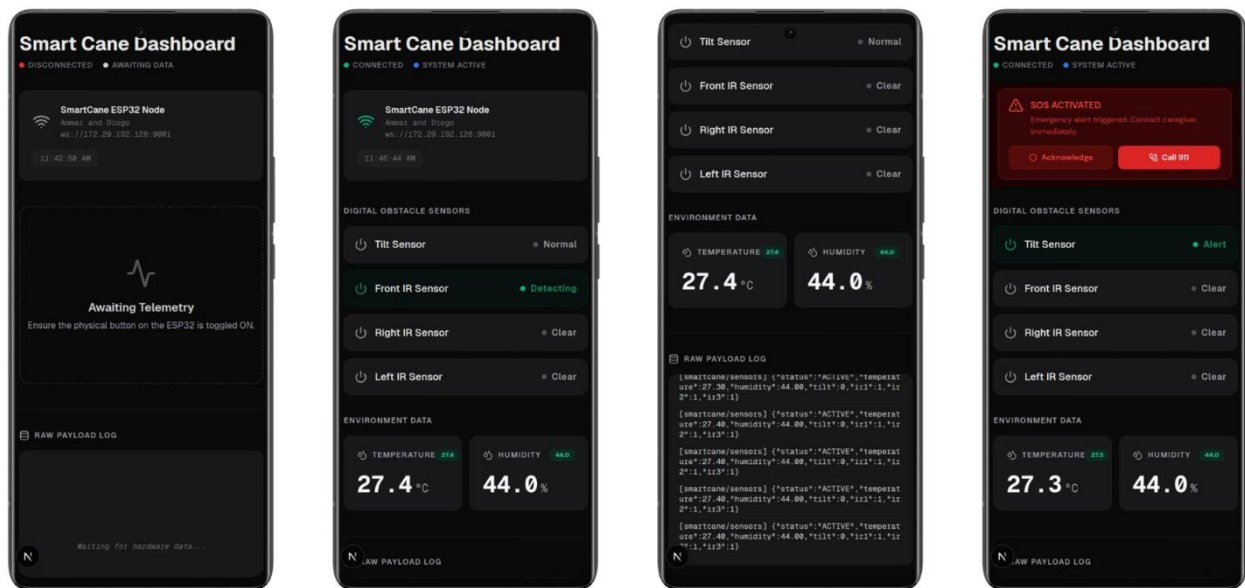


Image 12 – Frontend (smartphone)

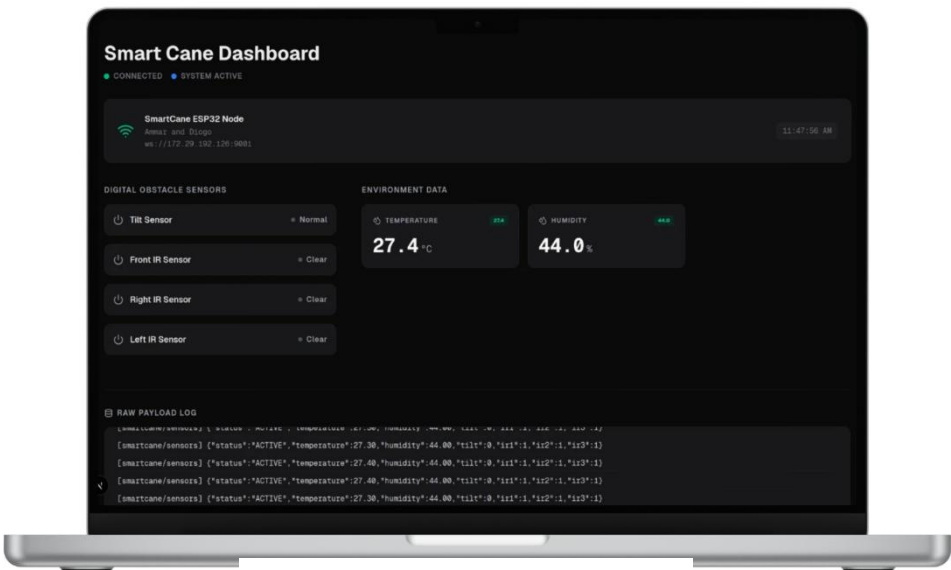


Image 13 – Frontend (computer)

6. Development Process

The project was executed in four distinct phases:

Phase 1: Software Infrastructure

We began by creating the data pipeline and frontend environment. Using Docker Compose, we deployed the Mosquitto MQTT broker alongside the Next.js frontend running on Bun. This made it possible for the WebSocket to communicate between them. Then, we build the React UI using mock JSON payloads before any hardware was involved.

Phase 2: ESP32 Networking

The second phase focused on the microcontroller's connectivity. We wrote the base firmware in the Arduino IDE to connect the ESP32 to the local Wi-Fi network and create a connection to the Mosquitto broker on port 1883 and implemented auto-reconnect.

Phase 3: Breadboard Prototype

Before finalizing the physical design, we wired all components (DHT11, three IR sensors, tilt switch, and buzzer) on a breadboard. This phase was needed and very important given we had to test individual sensor pinouts and validating digital/analog reads, calibrate the IR sensors for optimal obstacle detection distance, fine-tuning the buzzer frequencies (500Hz, 800Hz, 1000Hz) and delays to made the beep patterns easily distinguishable, and last but not least, this section of the project provided an excellent opportunity to gain hands-on experience in electronics.

Phase 4: Integration

The final phase transitioned the project from a breadboard to a functional prototype. The components were wired and secured inside a 3D printed enclosure. We made sure all the sensors were properly connected and the IR sensors were angled correctly to detect front, left, and right obstacles.

All the code described in this report is available in our GitHub repository, which can be accessed through the following link:

<https://github.com/falcaodiogo/PIOT-SoSe26.git>

7. Challenges and Limitations

During the development of the prototype, the timeframe did not allow for learning and implementing proper PCB soldering. While crimping provided a good enough alternative and a great learning experience, the lack of soldered connections dictated the physical constraints of the prototype. Because we had to rely on bulkier wiring methods rather than a compact soldered board, space and implementation constraints had to be prioritized. Following this, planned features like vibration motors, indicator LEDs, and dedicated physical power or SOS buttons had to be excluded. Also, time constraints prevented fully optimizing the CAD model. A more refined design in Fusion 360 would allow for completely internal cable routing, protecting the wiring from snagging, a more stable enclosure and improving the aesthetic finish of the PETG enclosure.

The selection of sensors also presented distinct limitations. For fall detection, the KY-017 tilt switch is a basic digital sensor that only detects movement in horizontal direction. It cannot differentiate between a genuine medical emergency, a dropped cane, or the user simply not using the device while it is powered on. While a 30-second timer was implemented to mitigate this, the risk of false positives remains high. For surface moisture detection, the KY-015 sensor measures ambient temperature and humidity. Relying on localized ambient humidity changes is too slow and indirect to reliably and instantly detect slippery surfaces or puddles in real-time. Regarding the infrared sensors used for obstacle detection, a major limitation was that they needed to measure a greater distance to be truly effective for the user's spatial awareness. Finally, the system currently relies on auditory-only feedback, which restricts its usability in noisy environments or for individuals with hearing impairments.

8. Future Improvements

To address the current limitations, future iterations of the Smart Cane would have to prioritize hardware miniaturization, sensor upgrades, and qualitative evaluation. Transitioning from crimped wiring to a custom Printed Circuit Board (PCB) would free up internal space, allowing for the reintroduction of initially planned components like vibration motors for haptic feedback and dedicated physical buttons. This miniaturization would also facilitate a more refined enclosure design in Autodesk Fusion 360, enabling completely internal cable routing to protect wires and improve the aesthetic finish. To have more reliability, the basic KY-017 tilt switch should be replaced with a 6-axis IMU to eliminate false positives in fall detection, while the infrared sensors could be upgraded to ultrasonic sensors to increase the range of obstacle detection. Moving beyond technical validation to structured user testing would be critical. Finally, an on/off button and a rechargeable battery should be added so that the cane can operate independently, removing the need for a constant connection to a laptop.

9. Conclusion

In conclusion, we are very satisfied with the work we've done. Although the Smart Cane may not be perfect, we think for the time window, we are satisfied with the results. We tried to use all the sensors available in the FABLAB, make a pretty-ish 3D printed enclosure and touch in every point in software that we could. Developing this Smart Cane allowed us to successfully combine hardware and software to build a tool that can actually help people. We learned a lot on this course in a wide variety of areas, and we are happy with the experience and the final result.

Bibliography

Curto Circuito. (n.d.). KY-017 datasheet. Available in:

<https://curtocircuito.com.br/datasheet/KY-017.pdf>

Handson Technology. (n.d.). Passive buzzer module. Available in:

<https://www.handsontec.com/dataspecs/module/passive%20buzzer.pdf>

Joy-IT. (2024, April 25). SEN-KY032IR manual. Available in: [https://www.joy-](https://www.joy-it.net/files/files/Produkte/SEN-KY032/SEN-KY032IR_Manual_2024-04-25.pdf)

[it.net/files/files/Produkte/SEN-KY032/SEN-KY032IR_Manual_2024-04-25.pdf](https://www.joy-it.net/files/files/Produkte/SEN-KY032/SEN-KY032IR_Manual_2024-04-25.pdf)

RCS Components. (n.d.). KY-015 datasheet. Available in:

<https://www.rcscomponents.kiev.ua/datasheets/ky-015-datasheet.pdf>

World Health Organization. (2024, January 2). Assistive technology. Available in:

<https://www.who.int/news-room/fact-sheets/detail/assistive-technology>